Aquí tienes los comandos exactos para hacer todo el flujo en **Git Bash**, desde la descarga hasta la subida a una **nueva rama**.

Como mencionas que el repositorio de Azure tiene usuario y clave (token), lo primero es preparar la autenticación para que no te dé errores.

🔐 Paso 0: Preparar la autenticación con Azure DevOps

Azure ya no acepta contraseñas normales. Necesitas un **Personal Access Token (PAT)**.

1. Ve a Azure DevOps → **User Settings** (arriba a la derecha) → **Personal Access Tokens**.
2. Crea uno nuevo con permisos de **Code (Read & Write)**.
3. Copia ese token. Lo usarás como "contraseña" cuando Git te lo pida.

📥 Paso 1: Clonar el repositorio

Abre **Git Bash** y ejecuta:

```bash
# 1. Ir a la carpeta donde quieres guardar el proyecto
cd /c/Users/TuUsuario/Documents/Proyectos

# 2. Clonar el repositorio (cambia la URL por la real)
git clone https://dev.azure.com/tu-empresa/tu-proyecto/_git/tu-repositorio

# 3. Entrar a la carpeta del repositorio recién descargado
cd tu-repositorio
```

(Cuando te pida usuario, pon tu correo. Cuando te pida contraseña, pega el Token que creaste).

🌿 Paso 2: Crear una nueva rama

Es importante **no trabajar en la rama principal (`main` o `master`)**. Vamos a crear una rama nueva y cambiarnos a ella:

```bash
# Ver en qué rama estás actualmente (debería decir main o master)
git branch

# Crear la nueva rama y cambiarte a ella automáticamente
git checkout -b feature/mi-nueva-pantalla
```

(Puedes cambiar `feature/mi-nueva-pantalla` por el nombre que quieras, sin espacios).

✏️ Paso 3: Editar los archivos

1. Abre tus archivos (los `.fmb`, `.sql`, `.pks`, etc.) con tu editor (Forms Builder, PL/SQL Developer, VS Code).
2. Haz todas las modificaciones que necesites.
3. Guarda los archivos.
4. Vuelve a Git Bash y verifica que Git detectó los cambios:

```bash
# Ver los archivos que modificaste (aparecerán en rojo)
git status
```

💾 Paso 4: Guardar los cambios (Commit)

Vamos a empaquetar los cambios para enviarlos:

```bash
# Agregar TODOS los archivos modificados al paquete
git add .

# O si quieres agregar solo uno en específico:
# git add forms/FOR_BLOQUEO_PV.fmb

# Crear el paquete con un mensaje descriptivo
git commit -m "feat: agregar nueva pantalla de bloqueos y scripts"
```

🚀 Paso 5: Subir la nueva rama a Azure DevOps

Aquí está el truco para que se suba a la **nueva rama** y no a la principal:

```bash
# Subir la rama nueva y decirle a Git que rastree esta rama (el -u es clave)
git push -u origin feature/mi-nueva-pantalla
```

¿Qué hace este comando?

- Sube tu rama `feature/mi-nueva-pantalla` a Azure.
- El `-u` (upstream) le dice a Git: "A partir de ahora, cuando yo haga `git push` o `git pull` sin especificar nada, hazlo sobre esta rama nueva".

🔍 ¿Cómo verifico que se subió bien?

1. Ve a la página web de tu repositorio en **Azure DevOps**.
2. En el menú de la izquierda, haz clic en **Branches** (Ramas).
3. Ahí deberías ver tu rama `feature/mi-nueva-pantalla` junto a la rama `main`.
4. Para unir tus cambios a la principal, por lo general en la empresa te pedirán crear un **Pull Request (PR)** desde tu rama hacia `main`. Puedes hacerlo directamente desde la web de Azure.

⚠ Reglas de oro para trabajar así

1. **Nunca** uses `git push origin main` si estás en tu rama nueva. Usa siempre `git push` (ya que el `-u` configuró todo).
2. Si por alguna razón Azure te rechaza el push, prueba con:
   `git pull origin feature/mi-nueva-pantalla` (por si alguien más tocó tu misma rama) y luego vuelve a hacer `git push`.
3. Para ver en qué rama estás parado en cualquier momento, escribe:
   `git status`
   La primera línea te dirá: "On branch feature/mi-nueva-pantalla".
   no quiero que se suban los archivos .fmx de oracle builder forms , solo los .fmb

1. **Analyze the User's Request**:

- **Goal**: Prevent `.fmx` files (compiled Oracle Forms binary files) from being uploaded to the Git repository on Azure DevOps. Only upload `.fmb` (source files).
- **Context**: The user is using Git Bash to push to an Azure DevOps repository. They just learned how to create a branch and push.
- **Needs**: A clear, step-by-step guide on how to configure Git to ignore specific file extensions (specifically `.fmx`).

2. **Determine the Best Solution**:

- The standard way to ignore files in Git is using a `.gitignore` file.
- If the `.fmx` files are already tracked by Git, simply adding them to `.gitignore` won't work; they need to be untracked (`git rm --cached`).
- Need to explain how to create the `.gitignore` file, what to put inside it, and how to handle files that might already be tracked.

3. **Formulate the Steps**:

- **Step 1: Check if `.fmx` files are already tracked.** (Crucial! If they are, `.gitignore` won't stop them).
- **Step 2: Remove `.fmx` from Git's tracking (without deleting local files).** Command: `git rm --cached *.fmx`.
- **Step 3: Create/Edit the `.gitignore` file.** Content: `*.fmx`, `*.mmx`, etc.
- **Step 4: Commit and push the changes.**

4. **Draft the Response (in Spanish, as requested/established)**:

- Greeting & Acknowledgment: Validate the practice. Uploading `.fmx` is bad practice (binary files, conflicts, space).
- Warning: Point out that if they already uploaded them, `.gitignore` alone won't work. They need to untrack them.
- Step 1: Create the `.gitignore` file. Use Git Bash commands (`touch .gitignore` or `echo "*.fmx" >> .gitignore`).
- Step 2: Untrack existing `.fmx` files. `git rm --cached *.fmx`. Explain that this doesn't delete them locally.

- Step 3: Commit the `.gitignore` and the untracked files. `git add .gitignore`, `git commit -m "chore: ignorar archivos compilados .fmx"`.
- Step 4: Push.
- Complete `.gitignore` template for Oracle Forms: Include `.fmx`, `.mmx`, `.plx`, `.rep`, `*.log`, etc.

5. **Refine the Explanation**:

- Make the Git Bash commands easy to copy-paste.
- Explain why (`git rm --cached` vs `git rm`).
- Anticipate edge cases (what if they have subfolders? `*.fmx` handles it recursively).

¡Excelente práctica! Subir archivos `.fmx` (que son binarios compilados) es un error muy común, ya que generan conflictos horribles en Git y ocupan espacio innecesario. Solo los `.fmb` (código fuente) deben versionarse.

Para lograr esto, debes usar un archivo llamado `.gitignore`.

Aquí tienes el paso a paso exacto. **Ojo:** Si ya habías subido archivos `.fmx` antes, no basta con crear el `.gitignore`; hay que decirle a Git que los "deje de rastrear".

🛑 Paso 1: Crear el archivo `.gitignore`

Abre tu **Git Bash**, asegúrate de estar en la carpeta de tu repositorio y ejecuta este comando para crear el archivo con las reglas:

```bash
# Crear el archivo .gitignore y agregar las reglas para Oracle Forms
echo "*.fmx" >> .gitignore
echo "*.mmx" >> .gitignore
echo "*.plx" >> .gitignore
echo "*.rep" >> .gitignore
```

(Puedes abrir el archivo `.gitignore` con el Bloc de notas para verificar que se creó. También puedes añadir ahí carpetas como `.vscode/` o `*.log`).

🖌️ Paso 2: Dejar de rastrear los `.fmx` (Si ya los habías subido)

Si en los pasos anteriores de tu trabajo ya hiciste `git add .` y subiste `.fmx`, Git los tiene "vigilados". Para decirle a Git que los olvide (sin borrarlos de tu disco duro), ejecuta:

```bash
# Quitar los .fmx del control de versiones (no se borran de tu PC)
git rm --cached *.fmx
```

(Si tienes los `.fmx` en carpetas dentro del proyecto, usa `git rm --cached -r *.fmx` o simplemente busca las rutas).

💾 Paso 3: Guardar y subir los cambios

Ahora que Git sabe que debe ignorar los `.fmx`, guardamos esta configuración y subimos:

```bash
# Agregar el archivo .gitignore y la eliminación de los .fmx
git add .gitignore
git add .

# Hacer el commit
git commit -m "chore: configurar .gitignore para evitar subir archivos .fmx"

# Subir a tu rama (recuerda usar tu nombre de rama)
git push
```

🧪 ¿Cómo comprobar que funcionó?

1. Compila un formulario en Forms Builder para generar un nuevo `.fmx`.
2. Ve a Git Bash y escribe:

```bash
git status
```

3. **El archivo `.fmx` NO debe aparecer** en la lista de archivos modificados (o aparecerá ignorado). Solo verás el `.fmb` si lo modificaste.

💡 Recomendación extra

Agrega el archivo `.gitignore` a la raíz de tu proyecto. Si quieres un `.gitignore` más completo para Oracle Forms, puedes abrirlo con el Bloc de notas y dejarlo así:

```
text
# Archivos compilados de Oracle Forms (NUNCA subir)
*.fmx
*.mmx
*.plx
*.rep
*.rdf

# Archivos temporales
*.tmp
*.log
*.bak

# Configuraciones de IDE
.vscode/
.idea/
```

dime como hacer un pase a otro servidior que lo hara otra persona. tengo los packgage con varios procedure creados, archivos con extension .pkg, . como se puede ejecutar el codigo del packege o se debe crear el archivo .sql script para ejecutar el packegae completo

The user is asking how to hand over their PL/SQL package code (in files with `.pkg` extension) to another person for deployment on a different server. They want to know if they can just run the `.pkg` file directly or if they need to create a `.sql` script.

I need to explain:

1. Oracle doesn't understand `.pkg` natively. It needs `.sql` or `.pls` scripts executed via SQL*Plus, SQL Developer, PL/SQL Developer, etc.
2. The `.pkg` extension is just a convention (often used by IDEs to separate the spec and body).
3. How to properly export/compile the package.
4. Step-by-step guide on what to deliver to the other person (the DBA/developer).
5. Best practices for deployment (permissions, compilation, validation).

Structure of the answer:

- Clear up the `.pkg` extension misconception.
- Explain how PL/SQL code is executed (it's just SQL script with `/` at the end).
- What files to deliver.
- Step-by-step deployment guide for the other person.
- Post-deployment validation.
- Common pitfalls (permissions, missing dependencies).

Let's draft the response in Spanish, keeping the tone helpful and professional.
El archivo con extensión `.pkg` **no es un formato que Oracle entienda de forma nativa**. Oracle solo entiende SQL y PL/SQL, y ese código debe ejecutarse a través de una herramienta cliente (como SQL*Plus, PL/SQL Developer o SQL Developer).

Ese `.pkg` es simplemente una convención de nombres que usan algunos IDEs (como SQL Developer) para separar la **especificación** (spec) del **cuerpo** (body) del paquete, pero por dentro es texto plano.

Para pasárselo a otra persona y que lo ejecute en otro servidor, **no necesitas inventar nada raro, solo empaquetar todo en un archivo** `.sql`.

Aquí tienes el paso a paso exacto para hacer el traspaso y cómo la otra persona debe ejecutarlo.

📦 Paso 1: Preparar el archivo para el traspaso (Lo que haces tú)

Si tienes el código en archivos `.pkg`, ábrelos con el Bloc de Notas (o tu editor) y asegúrate de que tengan la estructura estándar de Oracle.

Para entregar un paquete completo, debes juntar **dos cosas en un solo archivo** `.sql`:

1. El `CREATE OR REPLACE PACKAGE ...`
2. El `CREATE OR REPLACE PACKAGE BODY ...`

Crea un archivo nuevo llamado `PKG_MI_PAQUETE_DEPLOY.sql` y pega el código con este formato exacto (no olvides las barras `/` al final de cada bloque):

```sql
-- ============================================================================
-- SCRIPT DE DESPLIEGUE: PKG_PROCESOS_BLOQUEOS
-- Autor: [Tu Nombre]
-- Fecha: [Fecha]
-- ============================================================================

-- 1. ESPECIFICACIÓN DEL PAQUETE
CREATE OR REPLACE PACKAGE MI_ESQUEMA.PKG_PROCESOS_BLOQUEOS AS
    PROCEDURE PR_BLOQUEAR_PV(p_id_pv IN NUMBER);
    PROCEDURE PR_DESBLOQUEAR_PV(p_id_pv IN NUMBER);
END PKG_PROCESOS_BLOQUEOS;
/

-- 2. CUERPO DEL PAQUETE
CREATE OR REPLACE PACKAGE BODY MI_ESQUEMA.PKG_PROCESOS_BLOQUEOS AS
    PROCEDURE PR_BLOQUEAR_PV(p_id_pv IN NUMBER) IS
    BEGIN
        -- Tu código aquí
        NULL;
    END PR_BLOQUEAR_PV;

    PROCEDURE PR_DESBLOQUEAR_PV(p_id_pv IN NUMBER) IS
    BEGIN
        -- Tu código aquí
        NULL;
    END PR_DESBLOQUEAR_PV;
END PKG_PROCESOS_BLOQUEOS;
/
```

> **Nota importante:** Si el paquete lo va a instalar en un esquema diferente al tuyo, quita el prefijo `MI_ESQUEMA.` (o cámbialo por el esquema destino). Lo ideal es crear el paquete sin esquema y que el DBA lo ejecute estando conectado con el usuario correcto.

📂 Paso 2: ¿Qué más debes entregar? (El paquete completo)

No entregues solo el archivo del paquete. Para que el pase sea exitoso, entrégale un "combo" con estos archivos:

1. `01_TABLAS_Y_SECUENCIAS.sql` (Si creaste tablas nuevas, como `plazos` o `historial_bloqueo_pv`).
2. `02_VISTAS.sql` (Si usaste vistas como `V_PV_BLOQUEO`).
3. `03_PKG_MI_PAQUETE_DEPLOY.sql` (El que creamos arriba con el paquete).
4. `04_GRANTS.sql` (Los permisos).
5. `README.txt` (Un archivo de texto explicando qué hay que hacer).

En el archivo `04_GRANTS.sql` debes incluir los permisos para que el paquete funcione:

```sql
-- Si el usuario de la aplicación es diferente al dueño del paquete
GRANT EXECUTE ON MI_ESQUEMA.PKG_PROCESOS_BLOQUEOS TO USUARIO_APP;
```

🚀 Paso 3: Cómo lo ejecuta la otra persona (El pase)

La persona encargada del servidor destino debe recibir la carpeta con los `.sql` y hacer lo siguiente:

1. **Abrir PL/SQL Developer** (o SQL Developer).
2. **Conectarse con el usuario correcto** (el dueño del esquema, ej. `MI_ESQUEMA`).
3. **Abrir una "Command Window"** (Ventana de Comandos) o una "SQL Window".
4. Abrir el archivo `03_PKG_MI_PAQUETE_DEPLOY.sql`.
5. **Ejecutar todo el script:**

- En **SQL Developer**: Seleccionar todo el código (`Ctrl+A`) y presionar el botón de "Run Script" (`F5`).
- En **PL/SQL Developer**: Seleccionar todo el código (`Ctrl+A`) y presionar `F8` (o mejor aún, usar la Command Window para que ejecute las barras `/` sin problema).

6. Si la herramienta dice "Package created" y "Package Body created", ¡listo!

🔍 Paso 4: Validación (Lo que debe hacer la otra persona para confirmar)

Para que no haya sorpresas, pídele que ejecute estas dos consultas en el servidor destino para validar que el paquete quedó bien instalado:

```sql
-- 1. Verificar que el paquete y el body estén "VALID"
SELECT object_name, object_type, status
  FROM user_objects
 WHERE object_name = 'PKG_PROCESOS_BLOQUEOS';

-- 2. Verificar que no haya errores de compilación
SELECT line, position, text
  FROM user_errors
 WHERE name = 'PKG_PROCESOS_BLOQUEOS'
 ORDER BY sequence;
```

Si la primera consulta devuelve `VALID` en `PACKAGE` y `PACKAGE BODY`, y la segunda no devuelve ninguna fila, el pase fue un éxito.

⚠ Cosas críticas que debes advertirle a la otra persona

1. **Orden de ejecución:** Primero las tablas, luego las vistas, luego los paquetes, y al final los `GRANT`. Si ejecuta el paquete antes de crear las tablas, fallará.
2. **Dependencias:** Si tu paquete usa otras tablas existentes (como `punto_venta` o `cliente`), asegúrate de decirle que esas tablas deben existir en el servidor destino.
3. **Sinónimos:** Si tu código usa `@mssqldb` (el database link para SQL Server), pregúntale si ese link ya existe en el servidor destino. Si no, el paquete fallará al compilar o ejecutar.
4. **Barras `/`:** Insistirle que, si lo ejecuta con `F8` en una ventana normal, debe seleccionar todo el texto para que tome las barras `/` como separadores. Si no, solo compilará la especificación y el body quedará inválido.

💡 Resumen

No entregues el `.pkg`. Crea un solo archivo `.sql` que contenga la **Especificación**, luego el `/`, luego el **Body**, luego el `/`. Júntalo con los scripts de tablas y permisos, y pásale las instrucciones para que lo ejecute todo de una vez.